

Question:

Fundamentals of Object Oriented Programming: Object oriented paradigm, Object and Classes, Data Abstraction and Encapsulation, Inheritance, Polymorphism, Dynamic Binding.

1. Sample Case study on class & objects CO1

Bank Account Management System:

Problem Statement: Design a system to manage bank accounts.

Classes: Account, Savings-Account, Current-Account.

Attributes: Account number, balance, account type, interest rate (for savings account), overdraft limit (for current account), etc.

Methods: Deposit, Withdraw, Calculate Interest, Transfer, etc.

Object Usage: Create objects for each customer's account and perform transactions like deposits, withdrawals, and transfers.

steps :

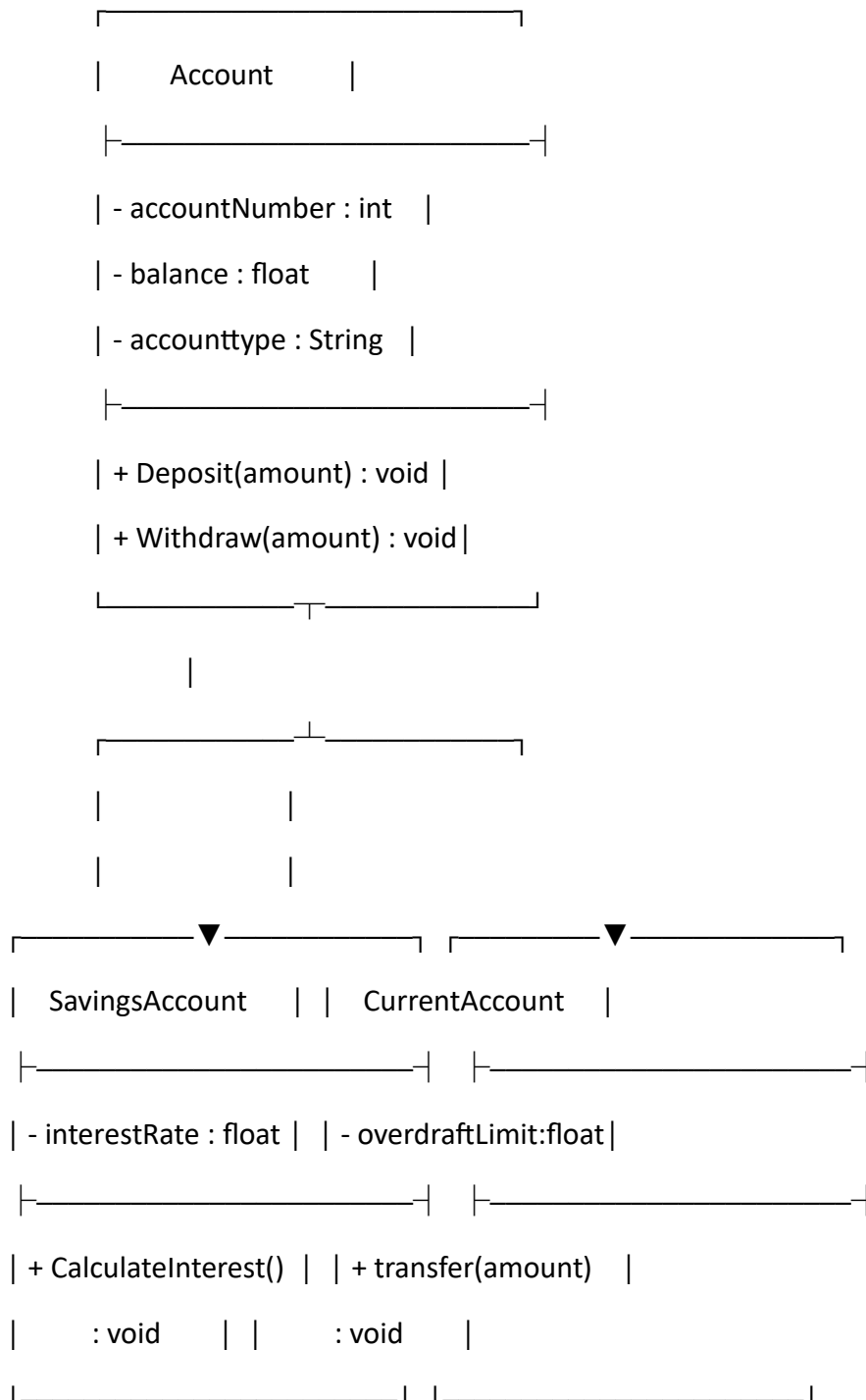
1. Create a superclass called Account.
2. Declare common account details:
 accountNumber
 balance
 accounttype
3. Create a Deposit() method to add money to the balance.
4. Create a Withdraw() method to withdraw money if sufficient balance is available.
5. Create a subclass SavingsAccount that inherits Account.
6. Add interestRate to SavingsAccount.
7. Create CalculateInterest() to calculate simple interest and update the balance.
8. Create another subclass CurrentAccount that inherits Account.

9. Add overdraftLimit to CurrentAccount.
10. Create transfer() to transfer money using the available balance and overdraft limit.
11. In main(), create a SavingsAccount object and assign its details.
12. Calculate the savings account interest.
13. Create a CurrentAccount object and assign its details.
14. Perform a transfer from the current account.
15. Display the account details and updated balances.

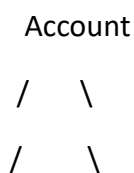
Algorithm :

1. Start
2. Create the Account class with account number, balance, and account type.
3. Define Deposit() method to add money to the account.
4. Define Withdraw() method to withdraw money if sufficient balance is available.
5. Create SavingsAccount class that inherits from Account.
6. Add interestRate and CalculateInterest() method to the SavingsAccount.
7. Create CurrentAccount class that inherits from Account.
8. Add overdraftLimit and transfer() method to the CurrentAccount.
9. In the main() method, create a SavingsAccount object.
10. Assign account number, balance, account type, and interest rate.
11. Calculate simple interest and update the savings account balance.
12. Display the savings account details.
13. Create a CurrentAccount object.
14. Assign account number, balance, account type, and overdraft limit.
15. Check whether the transfer amount is within the balance plus overdraft limit.
16. If the condition is satisfied, deduct the transfer amount and display the details.
17. Otherwise, display "Overdraft Limit Exceeded".
18. Stop.

UML Class Diagram :



Inheritance relationship





SavingsAccount CurrentAccount

code :

```
class Account {  
    int accountNumber;  
    float balance;  
    String accounttype;  
  
    void Deposit(float amount) {  
        balance = balance + amount;  
    }  
  
    void Withdraw(float amount) {  
        if (balance >= amount)  
            balance = balance - amount;  
        else  
            System.out.println("Insuffiecent Amount");  
    }  
}  
  
class SavingsAccount extends Account {  
    float interestRate;  
  
    void CalculateInterest(int time) {  
        float Simpleinterest = (balance * interestRate * time) / 100;  
        balance = balance + Simpleinterest;  
    }  
}
```

```

        System.out.println("Account Number : " + accountNumber);
        System.out.println("Account type : " + accounttype);
        System.out.println("Balance : " + balance);
        System.out.println("Simple Interest : " + Simpleinterest);
    }
}

class CurrentAccount extends Account {
    float overdraftLimit;

    void transfer(float amount) {
        if (balance + overdraftLimit >= amount) {
            balance = balance - amount;
            System.out.println("Account Number : " + accountNumber);
            System.out.println("Account type : " + accounttype);
            System.out.println("Transfer amount : " + amount);
            System.out.println("Current Account Balance : " + balance);
        } else {
            System.out.println("overdraft Limit Exceeded");
        }
    }
}

```

```

public class BankDemo {
    public static void main(String[] args) {
        SavingsAccount s1 = new SavingsAccount();
        s1.accountNumber = 546825370;
        s1.balance = 10000;
    }
}

```

```
s1.accounttype = "Savings";
```

```
s1.interestRate = 5;
```

```
s1.CalculateInterest(3);
```

```
System.out.println();
```

```
CurrentAccount c1 = new CurrentAccount();
```

```
c1.accountNumber = 128973968;
```

```
c1.balance = 20000;
```

```
c1.accounttype = "Current";
```

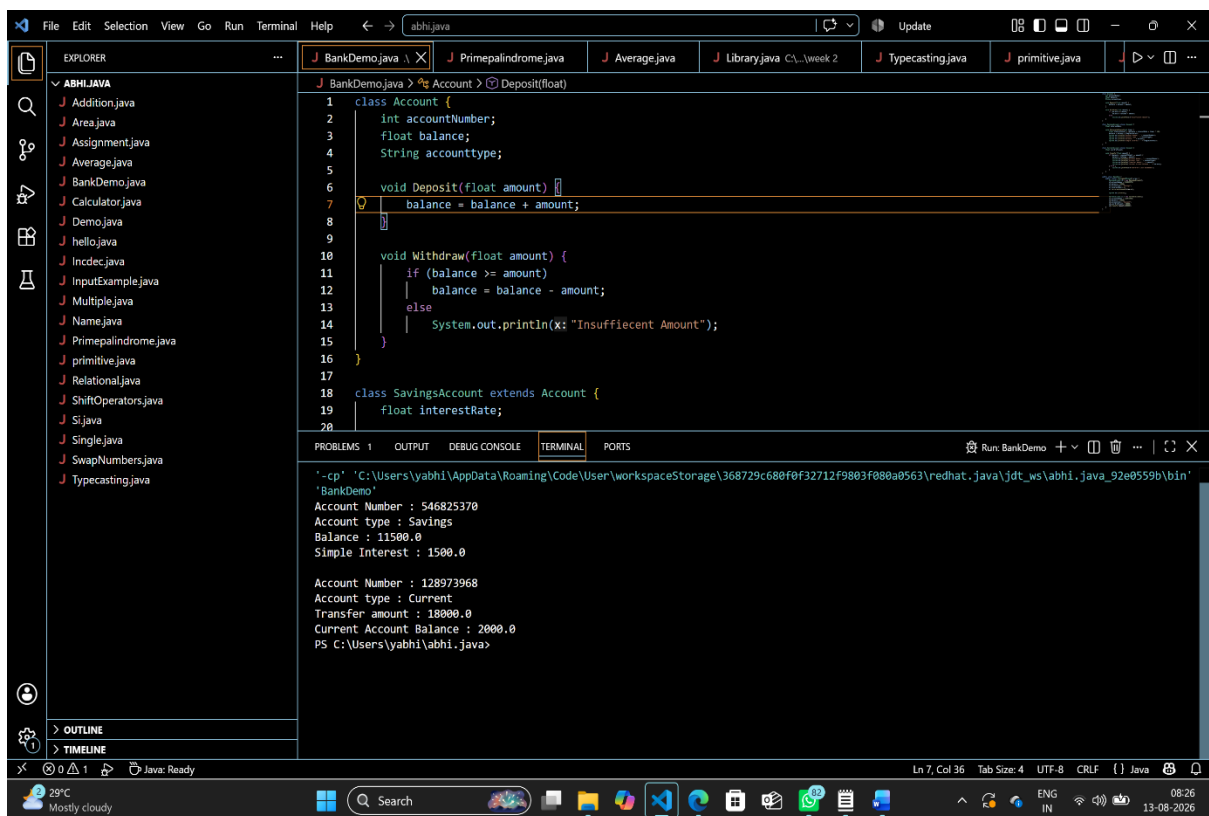
```
c1.overdraftLimit = 10000;
```

```
c1.transfer(18000);
```

```
}
```

```
}
```

Output :



The screenshot shows an IDE with a project named 'ABHIJAVA'. The Explorer panel on the left lists various Java files. The main editor displays the code for 'BankDemo.java', which defines an 'Account' class with methods 'Deposit' and 'Withdraw', and a 'SavingsAccount' class that extends 'Account'. The output window at the bottom shows the execution results of the program, including account details and the result of a transfer operation.

```
1 class Account {
2     int accountNumber;
3     float balance;
4     String accounttype;
5
6     void Deposit(float amount) {
7         balance = balance + amount;
8     }
9
10    void Withdraw(float amount) {
11        if (balance >= amount)
12            balance = balance - amount;
13        else
14            System.out.println("Insufficient Amount");
15    }
16 }
17
18 class SavingsAccount extends Account {
19     float interestRate;
20 }
```

Output:

```
PS C:\Users\yabhi\Abhi> java -cp ".\bin" BankDemo
Account Number : 546825370
Account type : Savings
Balance : 11500.0
Simple Interest : 1500.0

Account Number : 128973968
Account type : Current
Transfer amount : 18000.0
Current Account Balance : 2000.0
PS C:\Users\yabhi\Abhi>
```

